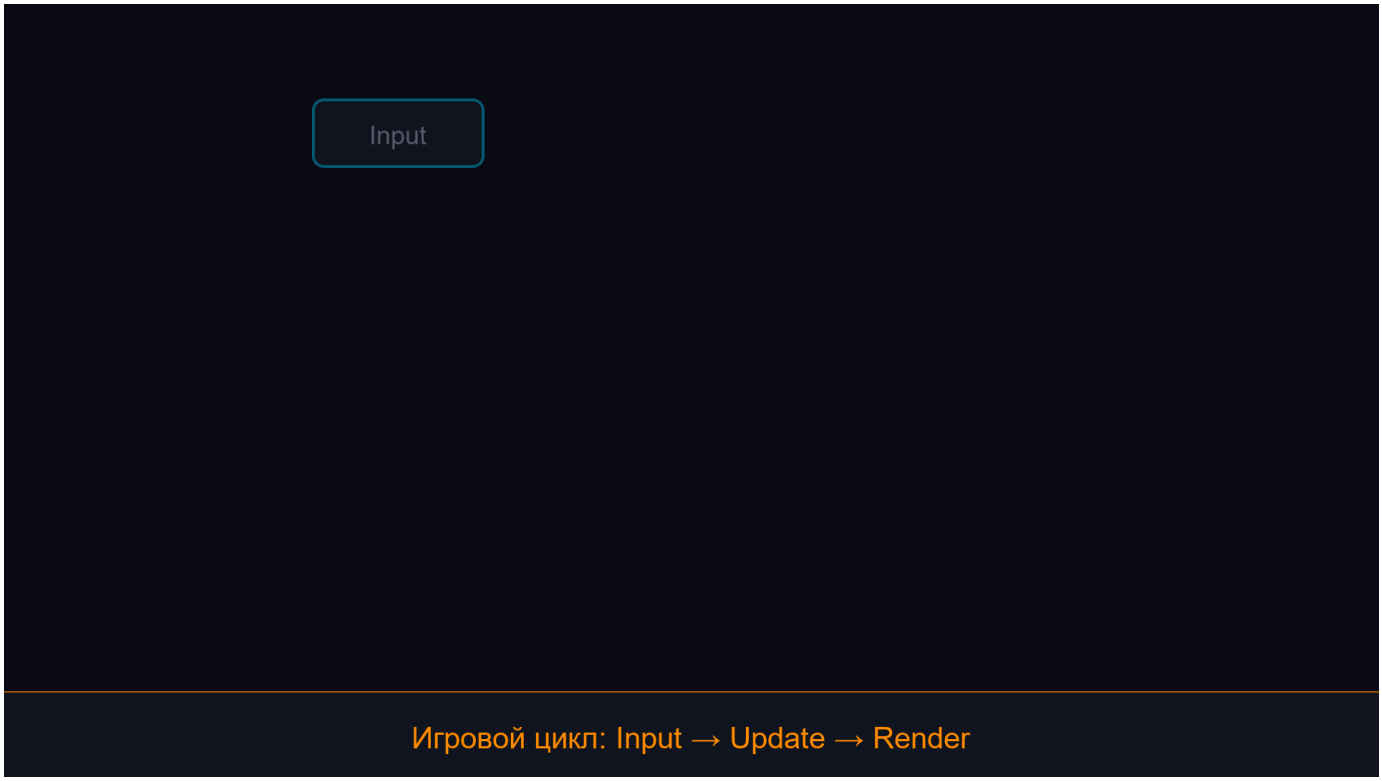
# Основной цикл

## Основной цикл




### Особенности игр

Прежде чем приступать к обзору (пусть и краткому) возможных архитектур и алгоритмов для игры, необходимо обговорить некоторые особенности работы игр.

Любая игра - это обычная программа, которая запускается на вашем компьютере и работает по тем же правилам, что и все остальные программы, построена на тех же принципах. Она может представлять из себя один большой exe-файл, запускающий один единственный процесс с одним единственным потоком. Или быть набором сотен файлов с ресурсами игры (текстурами, библиотеками и прочим), работающая в виде нескольких независимых процессов (“программ”), каждый из которых содержит в себе несколько параллельных потоков выполнения команд с множеством объектов, систем (по сути, набор объектов/сущностей) и сложными взаимосвязями между ними.

Несмотря на разнообразие подходов к разработке игры, можно выделить несколько распространенных идей, лежащих в основе любой игры.

### Основной Игровой Цикл

Основной цикл игры включает в себя обработку ввода пользователя, выполнение соответствующих действий, воздействие на игровой мир и его отрисовку.

Пример последовательности действий в игровом цикле:

1. Нажатие на кнопку “W”, которая ответственна за "Шаг вперед".
2. Проверка возможности выполнения шага вперед.
3. Перемещение персонажа (изменение его координат).
4. Перемещение объекта, в который персонаж уперся.
5. Отрисовка всего происходящего на экране.

Причем в теории, какие-то вещи мы можем делать параллельно, но для начала лучше разобраться с простыми, однопоточными вариантами. Так что эти действия хотелось бы выполнить в четком порядке, иначе все сломается. Если мы отрисуем сначала картинку, а потом передвинем персонажа, то для пользователя это будет выглядеть так, словно ничего не произошло, потому что мы... не отрисовали изменения.

### Про игровой цикл и кадры в игре:

## Время: Реальное и Игровое

Обработка ввода и других действий в игре может казаться мгновенной, но на самом деле требует времени. Компьютеры обрабатывают команды очень быстро, что позволяет играм создавать впечатление работы в реальном времени, а не в пошаговом режиме. Но, конечно, это команды все так же выполняются одна за другой.

Необходимо помнить о том, что ваш код компилируется в промежуточное представление (байткод) - “ассемблер” виртуальной машины C#, т.е. набор команд для процессора, а затем выполняется на этой виртуальной машине (ВМ). Что такое виртуальная машина в данном случае? Это обычная программа, которая выполняет роль некоторого универсального компьютера, что позволяет C# работать на большом количестве разных платформ. Так вот, ВМ компилирует при запуске ваш байткод в ассемблер физического процессора, на котором запускается данная ВМ. И вот процессор выполняет этот набор примитивных команд последовательно (не рассматриваем параллельность). Но делает это с огромной скоростью. Условно скажем - миллиарды команд в секунду.
Одна из тех вещей, что мы уже много раз поднимали на парах.

Для Python идея такая же в общих чертах.

Соответственно, программы, и игры в том числе, работают с некоторыми задержками, которые не заметны человеческому глазу. На самом деле, задержек гораздо больше - взять, к примеру, задержку изменения цвета пикселя на мониторе. Если она слишком большая, пользователь будет считать, что изображение двоится/троится/размывается.

Но если все работает пошагово, как делать анимации? Графику? ИИ? Давайте для начала упростим вопрос.

Как нам считать условную скорость персонажа? В пикселях в секунду? В командах процессора в секунду? Не совсем. Можно ввести дополнительные временные шкалы, которые частично будут "привязаны" к CPU.

1. **Оптимизация обновлений**

Нам не нужно обновлять каждое мгновение абсолютно все данные. Если никакие объекты не сдвинулись, то зачем проверять столкновения объектов? Если мы работаем с сетью, то мы не можем каждое мгновение отправлять и получать данные. Сигналу надо пройти тысячи километров, потратить на это время, поэтому мы получаем и отправляем данные, условно, только раз в секунду. И т.д. Поэтому некоторые системы, которые будут в вашей игре будут выполнять обновление каждую итерацию игрового цикла, другие - каждые три итерации и т.д.

2. **Приостановка игрового процесса**

Иногда нам необходимо приостановить игру. Но это не означает приостановить всю программу, подразумевается приостановка игрового процесса: монстры не атакуют, внутриигровое время не идет, персонаж не двигается, но вы можете нажимать на кнопки меню, к примеру.

3. **Измерение времени и скорости**

Мы хотели бы измерять, к примеру, скорость персонажа в метрах в секунду. Но эта секунда не должна зависеть от того, сколько кадров мы нарисовали, сколько времени реального времени прошло. Или же, наоборот, внутриигровая секунда должна совпадать с реальной секундой, вне зависимости от того, мощный или слабый у вас компьютер.

## Внедрение внутриигровой временной шкалы

- **Шкала времени A**: Универсальная шкала, соответствует реальному времени. Можно запросить время у процессора. Обычно, игровой движок берет это на себя.
- **Шкала времени B**: Внутриигровая шкала, позволяющая установить, что одна секунда по времени A равна двум секундам по времени B.

Использование шкалы B для всех игровых расчетов (скорости персонажа, времени и т.д.) и шкалы A для программных расчетов позволяет гибко управлять временем в игре, включая возможность его замедления, ускорения или полной остановки.

Как-то так:

Python C#

```python
import pygame
import sys

class Game:
    def __init__(self):
        # Settings
        self.is_paused = False
        self.game_speed = 1.0
```

```
            self.delta_time = 0.016  # will be updated each frame from clock.t

        def toggle_pause(self):
            self.is_paused = not self.is_paused

        def set_game_speed(self, speed):
            self.game_speed = speed

        def update(self):
            if not self.is_paused:
                scaled_delta_time = self.delta_time * self.game_speed
                self.update_game_world(scaled_delta_time)
            else:
                self.update_paused_state()

        def update_game_world(self, scaled_delta_time):
            # All game logic here, such as moving objects, checking collisions,
            # Example:
            # self.player.update(scaled_delta_time)
            # self.enemy.update(scaled_delta_time)
            # Physics.update(scaled_delta_time)
            pass

        def update_paused_state(self):
            # Update game menu or other paused state interactions
            # Example:
            # menu.update()
            pass

        def draw(self, screen):
            screen.fill((0, 0, 0))
            # All rendering here
            # Example:
            # self.player.draw(screen)
            # self.enemy.draw(screen)
            # self.ui.draw(screen)

    if __name__ == "__main__":
        pygame.init()
        screen = pygame.display.set_mode((800, 600))
        clock = pygame.time.Clock()
        game = Game()

        running = True

        # classic game loop: Input -> Update -> Render
        while running:
            # 1. Input
            for event in pygame.event.get():
                if event.type == pygame.QUIT:
                    running = False
```

```
            elif event.type == pygame.KEYDOWN:
                if event.key == pygame.K_p:
                    game.toggle_pause()

        # 2. Update
        game.update()

        # 3. Render
        game.draw(screen)
        pygame.display.flip()

        game.delta_time = clock.tick(60) / 1000.0  # seconds since last fra

    pygame.quit()
    sys.exit()
```

Мы можем ввести дополнительные локальные шкалы для анимаций. Т.е. начало анимации - это начало временной шкалы данной анимации. Это позволит ускорять, замедлять и останавливать анимации.

Но эти варианты не отвечают на вопрос, как нам создать иллюзию непрерывности игры. Ведь игра работает по кадрам, по шагам.

Если у нас игра - это один большой цикл, состоящий из итераций, мы можем сказать, что каждая итерация - это один кадр. Замерив время начала и конца итерации, получаем время которое потребовалось на отрисовку кадра - и это будет время, прошедшее с момента последнего обновления игры. И использовать его для расчетов при подготовке нового кадра (deltaTime в примере выше).

Ок. Но внутри `update` тоже есть структура: в каком порядке обновлять ввод, ИИ, физику, коллизии? Почему физике нужен фиксированный шаг? Что такое туннелирование? Подробнее - в разделе [Порядок обновления и физический цикл](#).

Теперь, когда мы понимаем, как устроен цикл, можно разобраться, как организовать код *внутри* него - этим занимается [Архитектура](#).

| Что | Где подробнее |
|---|---|
| **Следующий шаг: архитектура проекта** | **Архитектура** |
| Порядок систем и фиксированный шаг физики | Порядок обновления |
| deltaTime в анимациях | Анимации |
| Скорость без deltaTime - типичная ошибка | Типичные ошибки |
| Состояния игры (меню, пауза, геймплей) | Конечные автоматы |

| Что | Где подробнее |
| --- | --- |
| Коллизии и их проверка в цикле | Коллизии |
| Сетевой тик и частота обновлений | Сетевая архитектура |

Powered by Wiki.js